

宁波大学信息学院

实验报告

实验课程名称： 嵌入式系统

学　　院： 信息科学与工程学院

专　　业： 计算机科学与技术

指 导 教 师： 王翀,王艺睿

报告人姓名： 顾凯琪 **学号：** 226001624

班　　级： 2022 级计算机 2 班

学　　期： 2024-2025 学年第二学期

信息学院实验中心制

实验								
成绩								

实验项目名称	ARM 汇编实验（ADD）及 ARM 伪指令实验

一、实验目的与要求：

ARM 汇编实验（ADD）实验：

- 掌握 ARM 汇编语言的基本使用；
- 熟悉 eclipse 开发工具建立汇编工程和仿真；

ARM 伪指令实验：

- 掌握 ARM 汇编语言的基本使用和一些伪指令的使用；
- 熟悉 eclipse 开发工具建立汇编工程和仿真；

二、实验设备：

1. FS-JTAG 仿真器、USB 转串口线、电源
2. PC 机（OS：Windows 7）
3. Eclipse for ARM
4. GCC 编译工具
5. Yagarto 工具包
6. JRE
7. Exynos4412 实验箱

三、实验方法（原理、流程图）

ARM 汇编实验（ADD）实验：

原理：

根据实验指导书中阐述 RAM 汇编语言的使用语法和功能，编写汇编程序，实现一个简单的加法操作。

流程：

1. 导入光盘实验源码
   |---> A. 导入 01-add_asm 文件夹并编译程序
   |---> B. 配置 FS-JTAG 调试工具与 Eclipse 调试工具

2. 单步仿真调试
   |---> 运行程序，观察 Rn 寄存器变化

程序流程：

1. 开始
2. 设定常数值到寄存器 r0（mov r0, #9）
3. 设定常数值到寄存器 r1（mov r1, #15）
4. 将 r1 和 r0 相加，并存储结果到 r1（add r1, r1, r0）
5. 结束

ARM 伪指令实验：

原理：

根据实验指导书中阐述 RAM 汇编语言的使用语法和功能，编写汇编程序，实现将存放在两个内存中的数据相加的操作。

流程：

1. 导入工程源码和仿真测试

|---> A. 导入 03_sum 文件夹并编译程序

|---> B. 配置 FS-JTAG 调试工具与 Eclipse 调试工具

2. 单步仿真调试

|---> 运行程序，观察 Rn 寄存器变化

程序流程：

1. 开始
2. 将 myarray 地址加载到寄存器 r0（ldr r0, =myarray）
3. 初始化累加器寄存器 r4（mov r4, #0）
4. 循环开始（loop）

4.1 从 r0 指向的地址加载一个字到寄存器 r1，并将 r0 指针向前移动一个字（ldr r1, [r0], #4）

4.2 将加载的值（r1）加到累加器寄存器 r4 上（add r4, r4, r1）

4.3 比较加载的值（r1）是否为 0（cmp r1, #0）

4.4 若不为 0，则跳回循环开始处；若为 0，则终止循环

5. 无限循环（stop）

四、实验过程、步骤及内容

说明：部分实验图标注日期为 2014 年，为实验指导书中清晰示例图，仅为丰富实验指导书内容所置，不影响实验报告撰写。

ARM 汇编实验（ADD）实验：

实验内容：

1、 汇编加法程序设计

2、 ASM Code

```
    .globl  _start
_start:
     mov r0,#9 //9 存入 r0
    mov r1,#15    //15 存入 r1
    add r1,r1,r0//r1 与 r0 相加存入 r1
stop:
    b stop
```

实验步骤：

1、 导入光盘实验源码

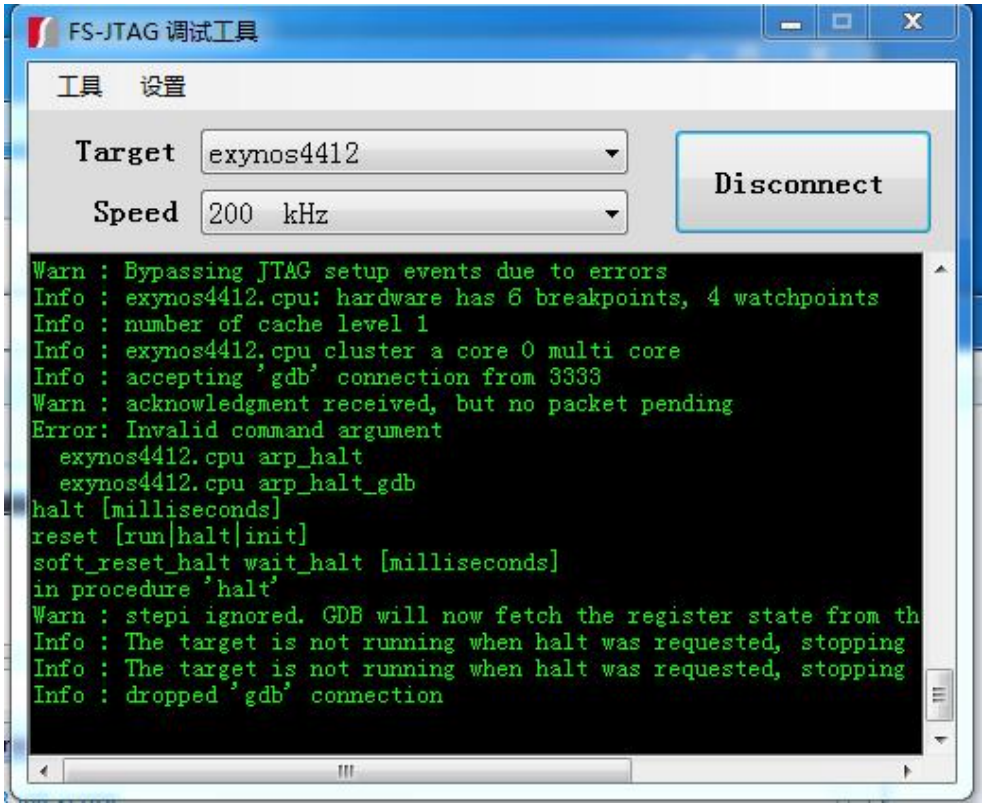
方法参考第 1 章 ARM 环境搭建。光盘实验源码路径：华清远见-CORTEXA9 资料\实验代码\1. ARM 体系结构与接口技术部分\实验代码\01-add_asm，

注意：ARM 仿真需要安装 ARM 开发环境，参考第 1 章 ARM 环境搭建的部分。

要把它放到其他地方去，比如 D 盘中（路径中不可以有中文）。

A. 导入 01-add_asm 文件夹并编译程序

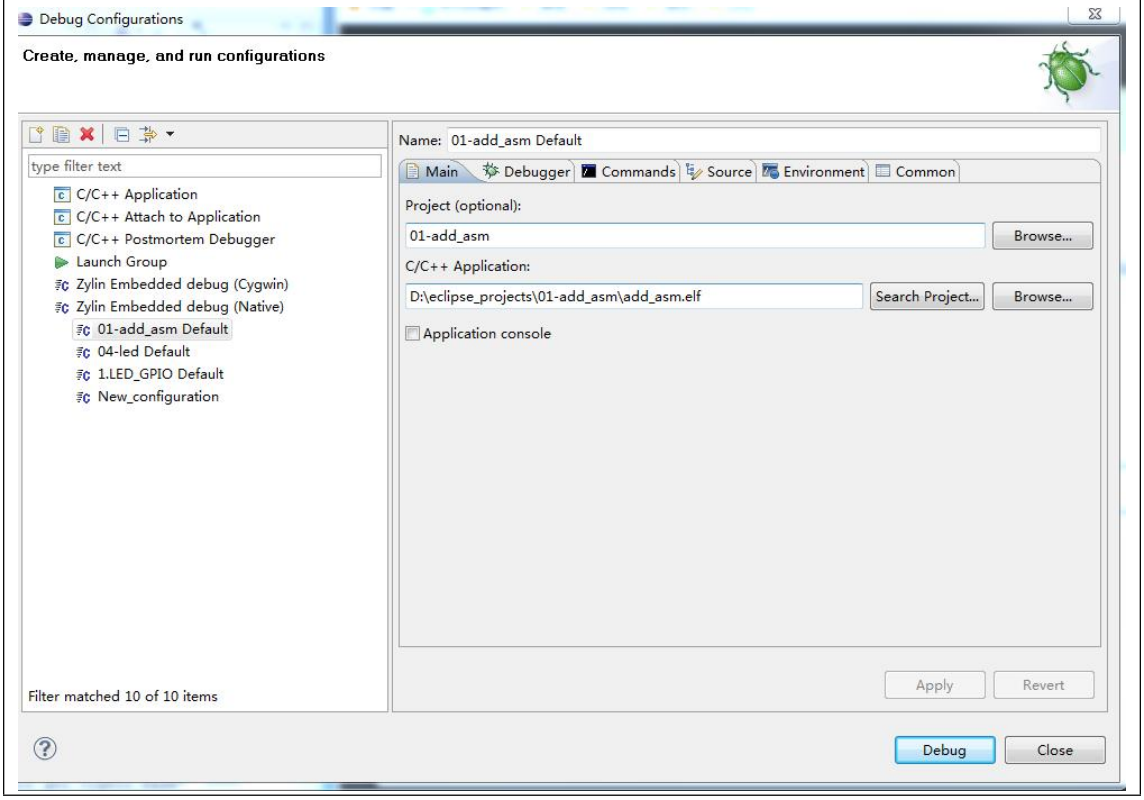
B. 配置 FS-JTAG 调试工具与 Eclipse 调试工具



选中“Run”→“Debug Configuras…”

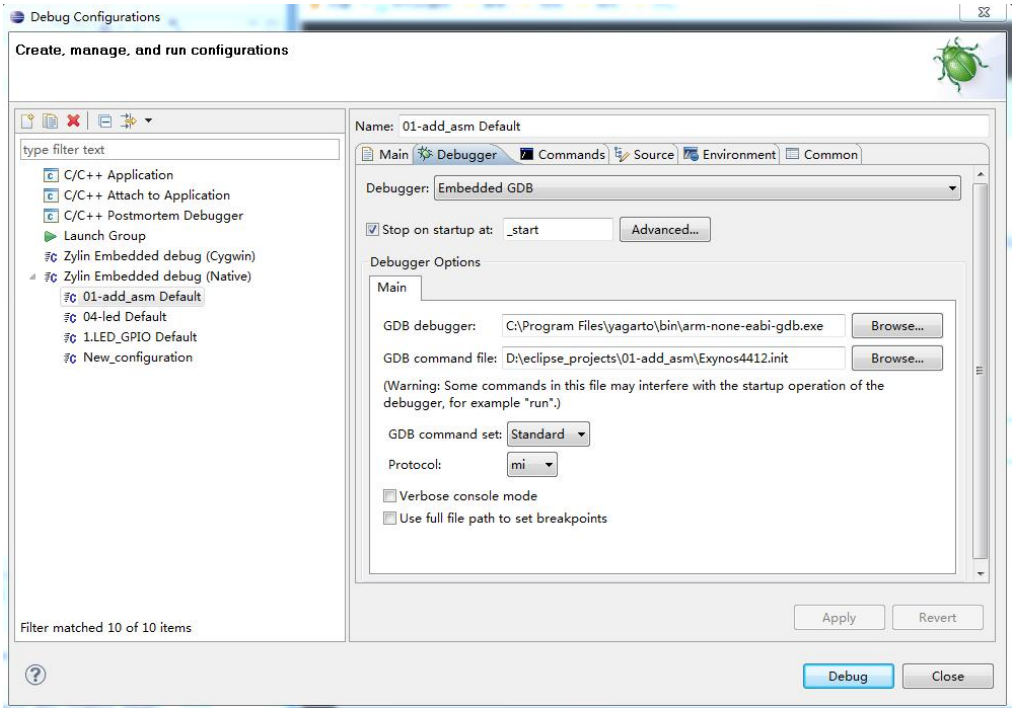
在“Zylin Embedded debug (Native)”选项上右击鼠标，在点击“New”选项

然后再点击“Main”选项栏中红色框内的“Browse”选项选择将要执行的程序，找到 01-add_asm 工程对应的目录，选中目录中的 **add_asm.elf** 文件，然后选择“打开”，如下图所示。



【Debugger 选项卡】

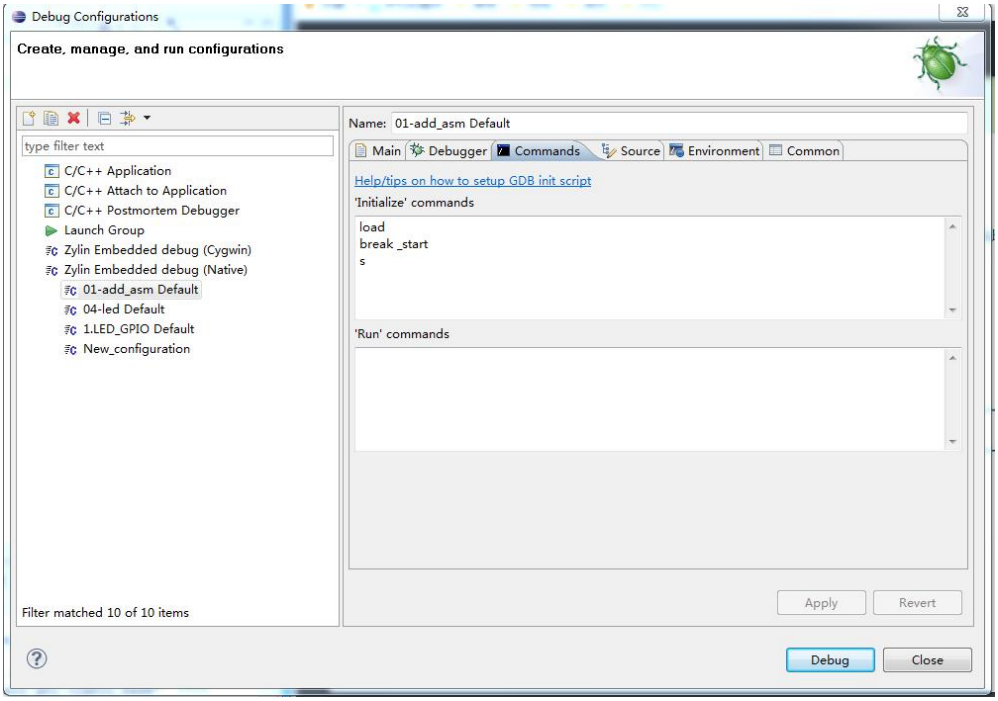
在 Debugger 选项卡中的 Main 子选项卡中的 GDB debugger 的框中单击“Browse”按钮选择前面安装的 arm-none-eabi-gdb.exe（这里选择自己安装 gcc 编译工具的安装目录 C:\Program Files\yagarto\bin），在 GDB Command file 中选择自己工程目录下的 Exynos4412.init 文件，如图所示。



【Command 选项卡】

在 Command 选项中，如下图所示，在“Initialize’ commands”中添加 3 行命令

```
load
break _start
s
```

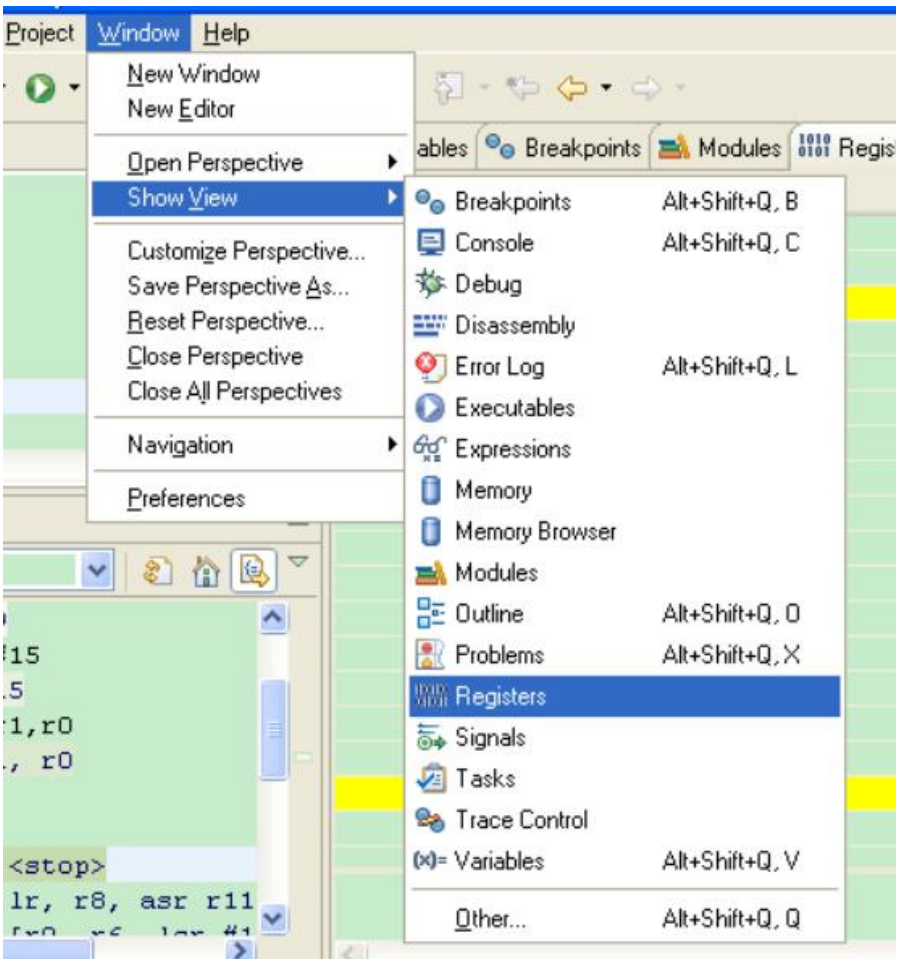


意思为从_start 开始，单步运行。

添加完后点击如下图所示的“Apply”，调试选项配置完成。再点击“Debug”选项进入就可以进行调试了。

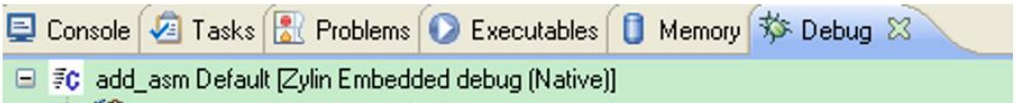
2、 打开“Register”显示框

单击 window -> show view -> Register，



3、 单步仿真

配置完成之后，点击“  ”开始仿真，弹出 Debug 框。



单击“  ”单步仿真。查看仿真现象，具体现象分析见**五、实验数据（现象）处理分析**。

ARM 伪指令实验：

实验内容：

汇编加法程序设计

```
    .text
.global _start
_start:
```

```
4           mov     r3,r3
5           ldr     r0,=myarray
6           mov     r4,#0
7   loop:
8           ldr     r1,[r0],#4
9           add     r4,r4,r1
10          cmp     r1,#0
11          bne     loop
12  stop:
13          b       stop
14  myarray:
15          .word   0x11
16          .word   0x22
            .word   0x0
            .end
```

这段代码定义了一个名为 myarray 的数组，其中包含两个非零元素（0x11 和 0x22）以及一个终止标记（0x0）。在主程序_start 中，使用循环将数组的所有元素加起来，并将结果保存在寄存器 r4 中。循环检查通过判断这些元素是否为 0 来终止。

简要步骤如下：

1、 程序开始并初始化指针 r0 指向数组 myarray。

2、 累加器（r4）初始化为 0。

3、 每次循环，加载当前 r0 指针所指的数组元素到 r1，并将 r0 的指针加 4（向前移动到数组的下一元素）。

4、 将加载的值（r1）加到 r4 上。

5、 检查 r1 是否为 0，即数组是否已经结束。如果不为 0，继续循环；否则退出。

6、 循环结束后，r4 寄存器将包含数组元素的总和。

7、 程序进入一个无限循环标签 stop

这样，寄存器 r4 最终包含的是 0x11 和 0x22 相加的和，即 0x11 + 0x22 = 0x33，及十进制数 51。

实验步骤：

1. 导入工程源码和仿真测试

参考第 1 章节的 ARM 开发环境搭建部分

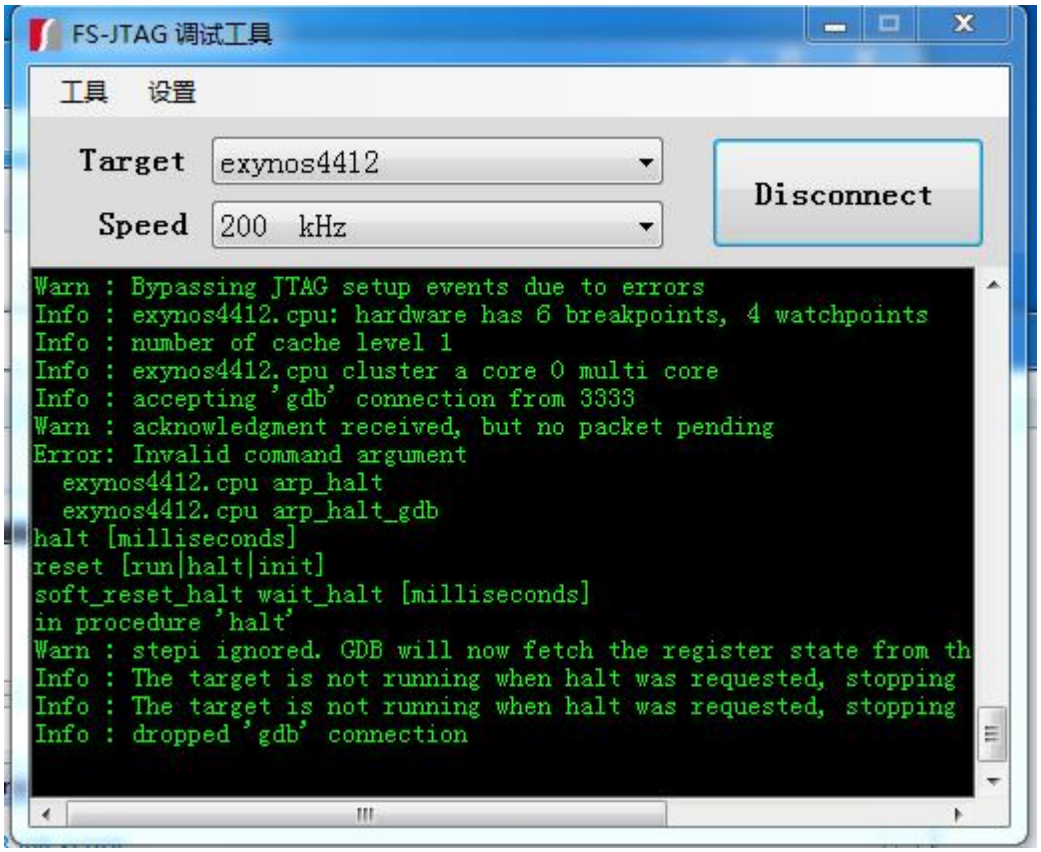
光盘实验源码路径：华清远见-CORTEXA9 资料\实验代码\1. ARM 体系结构与接口技术部分\实验代码\03-sum

注意：ARM 仿真需要安装 ARM 开发环境，请参考第 1 章节的 ARM 开发环境搭建部分。

要把它放到其他地方去，比如 D 盘中（路径中不可以有中文）。

A． 导入 03_sum 文件夹并编译程序

B． 配置 FS-JTAG 调试工具与 Eclipse 调试工具



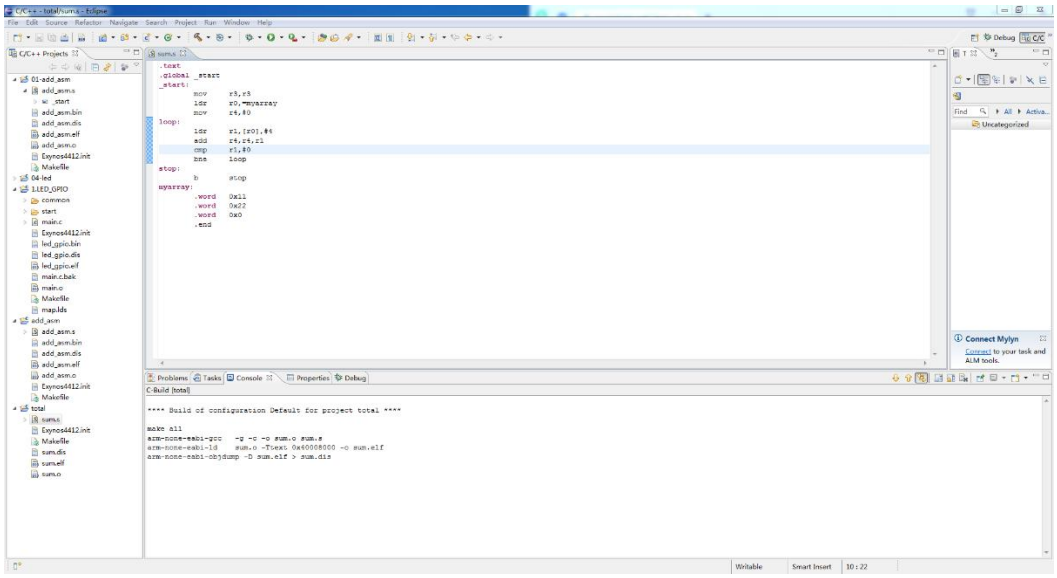
选中“Run”→“Debug Configuras…”

在“Zylin Embedded debug (Native)”选项上右击鼠标，在点击“New”选项

然后再点击“Main”选项栏中红色框内的“Browse”选项选择将要执行的程序，找到03-sum 工程对应的目录，选中目录中的 **sum.elf** 文件，然后选择“打开”，具体同 add 实验，Debugger 选项卡与 Command 选项卡的内容也近似于上面的 add 实验，本处不加以赘述。

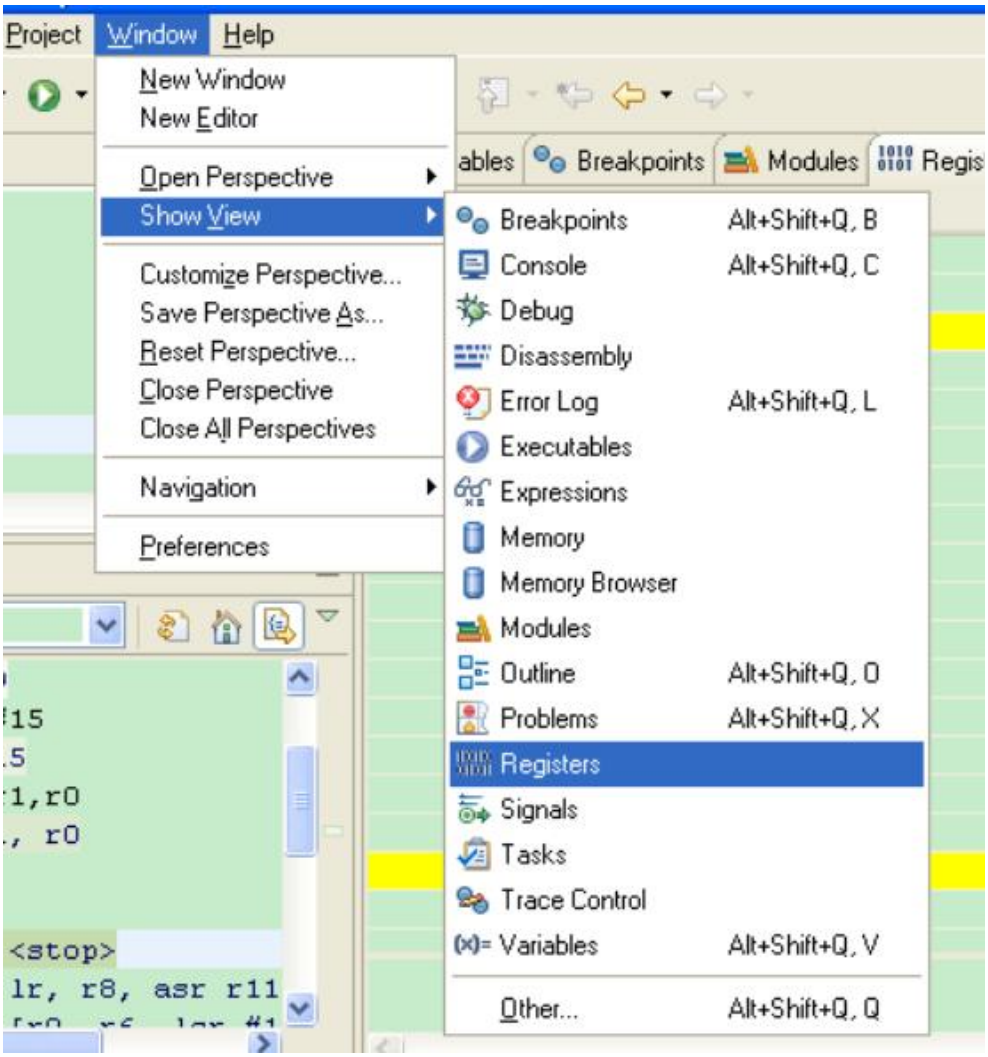
添加完后点击如“Apply”，调试选项配置完成。再点击“Debug”选项进入就可以进行调试了。

调试结果如下：



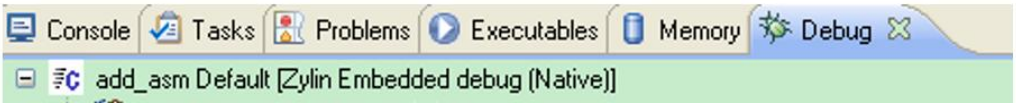
2. 打开“Register”显示框

单击 window -> show view -> Register，



3. 单步仿真

配置完成之后，点击“ ”开始仿真，弹出 Debug 框。



单击“ ”单步仿真。查看仿真现象，具体现象见**五、实验数据（现象）处理分析**。

五、实验数据（现象）处理分析

ARM 汇编实验（ADD）实验：

1、 单击“ ”单步，查看 Rn 寄存器的变化。

可以看到，运行完“mov r0,#9”的时候，寄存器中 r0 的值变为 9。

```
.global _start
_start:
    mov r0,#42
    mov r1,#6
    add r1,r1,r0
stop:
    b stop
```

Name	Value
Main	
r0	9
r1	0
r2	2146141320
r3	2146141320

可以看到，当运行完“mov r1,#15”的时候，寄存器中 r1 的值变为 15。

```
.global _start
_start:
    mov r0,#9
    mov r1,#15
    add r1,r1,r0
stop:
    b stop
```

Name	Value
Main	
r0	9
r1	15

可以看到，当运行完“add r1,r1,r0”（意思为把 r1 和 r0 的值相加，放到 r1 中）的时候，寄存器中 r1 的值变为 24，与预期相符。

```
.global _start
_start:
    mov r0,#9
    mov r1,#15
    add r1,r1,r0
stop:
    b stop
```

Name	Value
Main	
r0	9
r1	24

最后无条件跳转至 stop 标签，程序结束。

```
.global _start
_start:
    mov r0,#9
    mov r1,#15
    add r1,r1,r0
stop:
    b stop
```

Name	Value
Main	
r0	9
r1	24

ARM 伪指令实验：

单击“  ”单步，查看 Rn 寄存器的变化。

两个数据的和保存在 r4，最终 r4 的值为 0x33。

具体步骤：

将'myarray'的地址加载到寄存器 r0：

```
.text
.global _start
_start:
        mov     r3,r3
        ldr     r0,=myarray
        mov     r4,#0
loop:
        ldr     r1,[r0],#4
        add     r4,r4,r1
```

Name	Value
Main	
r0	1073774624
r1	0
r2	2146141320
r3	327155712
r4	0

将 0 移动到寄存器 r4，用来累加数组元素的和：

```
.text
.global _start
_start:
        mov     r3,r3
        ldr     r0,=myarray
        mov     r4,#0
loop:
        ldr     r1,[r0],#4
        add     r4,r4,r1
        cmp     r1,#0
        bne     loop
stop:
        b       stop
```

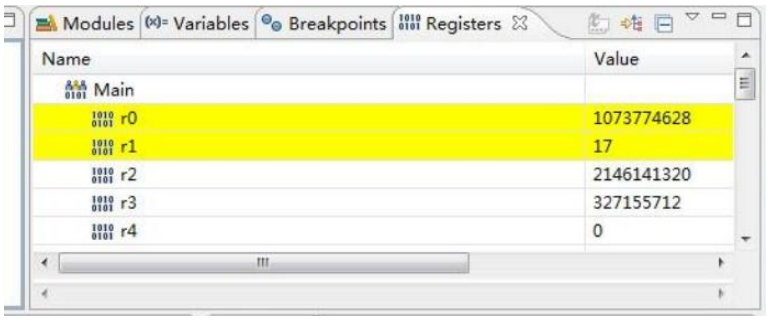
Modules | Variables | Breakpoints | Registers

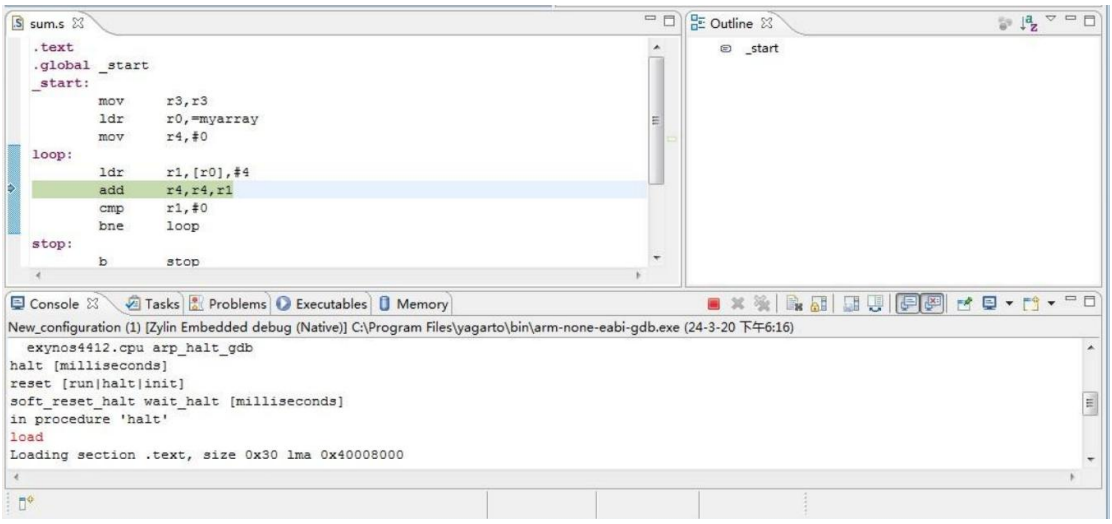
Name	Value
Main	
r0	1073774624
r1	0
r2	2146141320
r3	327155712
r4	0

loop 标签处：

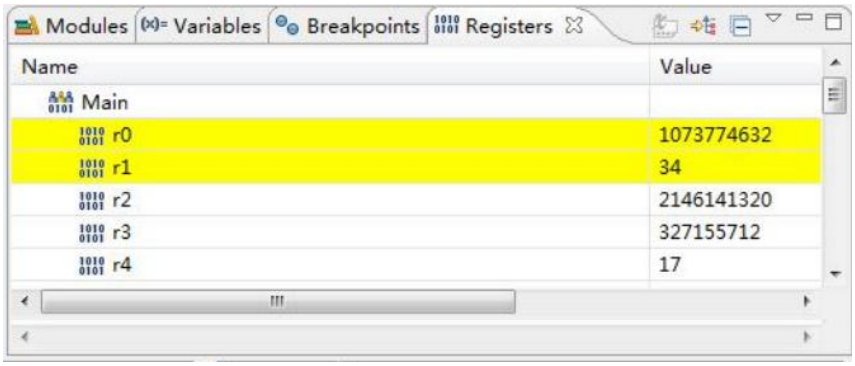
- **ldr r1, [r0],#4**：从 r0 指向的内存地址加载一个字(4 字节)到 r1 寄存器中，然后将 r0 的值增加 4。
- **add r4,r4,r1**：将 r4 寄存器中的值与 r1 寄存器中的值相加，结果存回 r4。
- **cmp r1,#0** ：将 r1 寄存器的值与 0 进行比较，但不会改变寄存器的值。
- **bne loop**：如果上一次比较结果不等于，则跳转到 loop 标签处。否则继续下一次循环

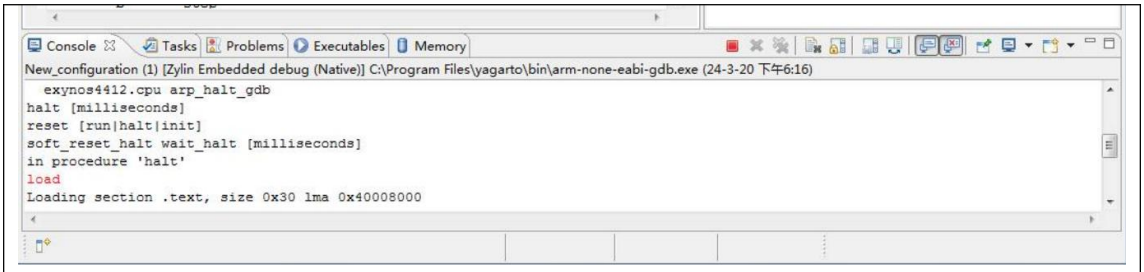
第一次循环：


Modules
Variables
Breakpoints
Registers
Name
Value
Main
r0
1073774628
r1
17
r2
2146141320
r3
327155712
r4
0

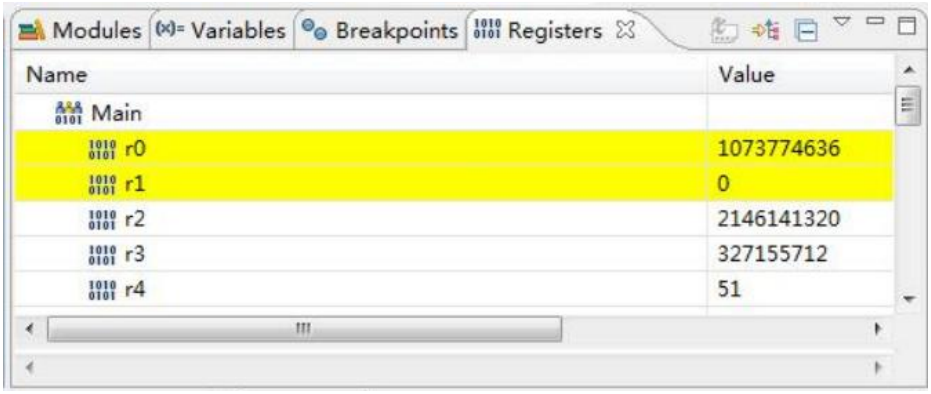

sum.s
.text
.global _start
_start:
mov r3,r3
ldr r0,=myarray
mov r4,#0
loop:
ldr r1,[r0],#4
add r4,r4,r1
cmp r1,#0
bne loop
stop:
b stop
Outline
_start
Console
Tasks
Problems
Executables
Memory
New_configuration (1) [Zylin Embedded debug (Native)] C:\Program Files\yagarto\bin\arm-none-eabi-gdb.exe (24-3-20 下午6:16)
exynos4412.cpu arp_halt_gdb
halt [milliseconds]
reset [run|halt|init]
soft_reset_halt wait_halt [milliseconds]
in procedure 'halt'
load
Loading section .text, size 0x30 lma 0x40008000

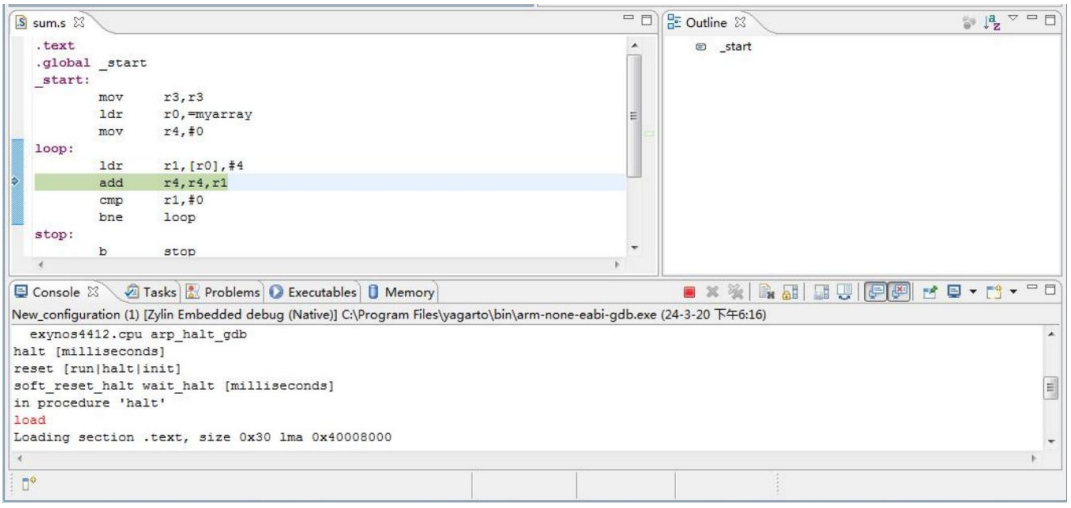
第二次循环：


Modules
Variables
Breakpoints
Registers
Name
Value
Main
r0
1073774632
r1
34
r2
2146141320
r3
327155712
r4
17



第三次循环：

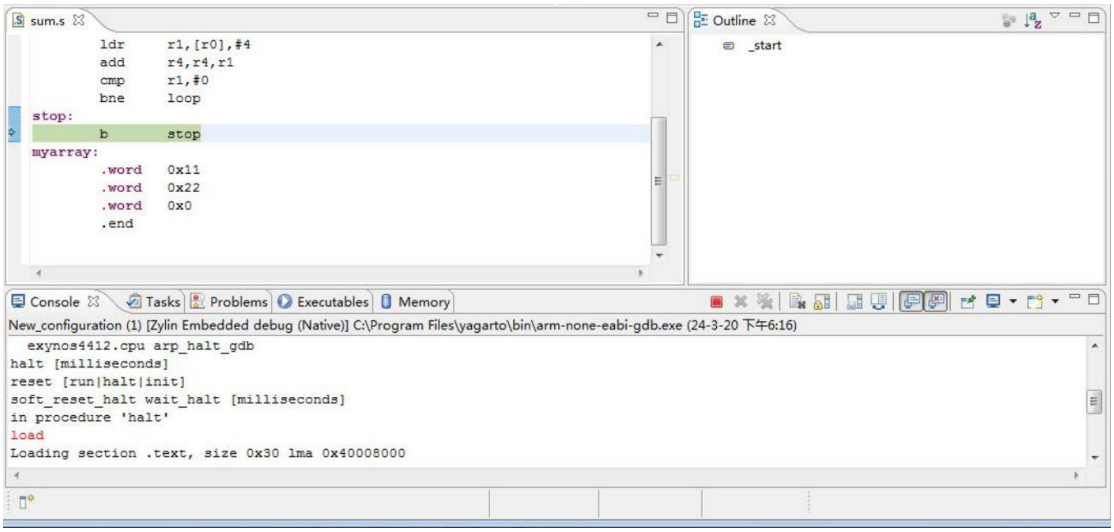




当遇到数值为 0 的元素时，程序跳转到 stop 标签处，然后又跳回到自身，形 成一个无限循环，最后 r1 结果为 0 ，r4 结果为 51。

stop 标签处，程序无限循环，不会改变任何寄存器的值。





思考题：

ADD 实验：

1、 用 ARM 汇编实现下面列出的操作：

1) r0=15

```
mov r0, #15
```

2) r0=r1/16（有符号数）

```
sxtb r0, r1    ; 有符号扩展 r1 的最低字节到整个 r0
asr r0, r0, #4 ; 算术右移 4 位，相当于除以 16
```

3) r1=r2*3

```
lsl r1, r2, #1 ; 左移一位，相当于乘以 2
add r1, r1, r2 ; 再加上一次 r2，相当于乘以 3
```

4) r0=-r0

```
rsb r0, r0, #0 ; 求补操作，相当于取反加一
```

2、 BIC 指令的作用是什么？

答：BIC（**Bitwise Bit Clear**）指令是 ARM 处理器中的一条按位清除指令，用于对一个寄存器的值进行按位 **AND 取反** 操作。其作用是将 **目标寄存器 Rn 中 Operand2 对应的位清零**，而不影响其他位，相当于 Rn & ~Operand2。

在实际应用中，BIC 指令常用于 **清除特定位**，例如掩码操作、寄存器位控制等。相比于 AND + MVN 的组合，BIC 指令执行效率更高，因为它可以在一个指令周期内完成 AND 取反操作。例如，BIC R1, R0, #0xF0 的作用是将 R0 变量中 **高 4 位清零**，低 4 位保持不变。

总的来说，BIC 指令在 ARM 汇编中是一种高效的 **位清除（Bit Clear）** 操作方式，广泛应用于 **寄存器控制、位掩码计算和优化位操作** 等场景。

```
BIC R0, R0, # 0x1F ; (0x1F=00011111b), 清除 R0 中的 bit [4:0] 位。
```

3、 执行 SWI 指令时会发生什么？

答：执行 SWI（软件中断）指令时，ARM 处理器会触发异常，并切换到 **超级用户模式（Supervisor Mode）** 处理该中断。

首先，处理器会将 **PC+4** 存入 LR_svc（链接寄存器），并将 CPSR（当前状态寄存器）保存到 SPSR_svc，然后切换到异常模式，并禁用 IRQ（硬件中断），确保异常处理不会被打断。

接着，处理器会跳转到异常向量表的 SWI 处理程序地址（通常是 **0x00000008**），执行相应的中断处理逻辑。在异常处理过程中，系统可以根据 SWI 指令携带的编号参数执行不同的服务，如 **系统调用、任务调度或特权级转换**。

执行完 SWI 处理程序后，处理器通常会通过 MOVS PC, LR 或 SUBS PC, LR, #4 指令恢复原程序的执行，继续运行用户代码。

4、 B、BL、BX 指令的区别是什么？

B、BL 和 BX 指令都是 ARM 处理器中的跳转指令，但它们在功能和使用场景上有所不同。

B 指令用于无条件跳转，直接修改 PC 寄存器的值，但不会保存返回地址，常用于循环或条件分支跳转。

BL 指令除了跳转外，还会将返回地址存入 LR（链接寄存器），用于函数调用，使得子程序执行完毕后可以通过 MOV PC, LR 返回调用点。

BX 指令用于跳转到寄存器地址，并根据目标地址的最低位决定是否切换到 Thumb 代码模式，常用于从子程序返回或者在 ARM 和 Thumb 指令集之间切换。

综上，B 适用于普通跳转，BL 适用于子程序调用，BX 适用于返回或指令集切换。

5、 下面哪个数据可以作为数据操作指令的有效立即数：

a．0x101 b．0x1f8 c．0xf000000f d．0x08000012 e．0x104

答：c．0xf000000f

理由如下：

a．0x101：十六进制数，如果作为立即数，可以被表示为一个 8 位的无符号数。在 ARM 指令集中，通常需要 12 位来表示立即数，所以 0x101 无法用一个指令来直接表示。

b．0x1f8：同上

c．0xf000000f：32 位的，包含了 ARM 指令集所能表示的范围。

d．0x08000012：这个立即数是 32 位的，但它太大了，超出了 ARM 指令集所能表示的范围。

e．0x104：同 a

6、 ARM 在哪些工作工作模式下可以修改 CPSR 寄存器？

答：在 ARM 处理器中，CPSR（**Current Program Status Register**，当前程序状态寄存器）存储了处理器的状态信息，包括条件标志、控制位和模式位等。由于 CPSR 影响处理器的整体运行状态，因此它的修改受到权限限制，只有在特权模式下才能进行修改。

ARM 处理器有 **7 种特权模式**，其中只有 **超级用户模式（SVC）、中断模式（IRQ）、快速中断模式（FIQ）、数据中止模式（ABT）、未定义指令模式（UND）** 和 **系统模式（SYS）** 能够修改 CPSR，而 **用户模式（USR）** 不能直接修改 CPSR。

在特权模式下，可以使用 MSR（Move to Status Register）指令来修改 CPSR，例如：

```
MSR CPSR_c, #0xD3  ; 设置 CPSR，切换到 SVC 模式并禁用中断
```

综上，ARM 处理器只有在 **超级用户模式（SVC）、中断模式（IRQ）、快速中断模式（FIQ）、数据中止模式（ABT）、未定义指令模式（UND）和系统模式（SYS）** 下，才能修改 CPSR，而在 **用户模式（USR）** 下则无法直接修改 CPSR。

7、 写一个程序，判断 R0 的值，大于 0x50，则将 R1 的值减去 0x10，并把结果送给 R0。

答：

```
cmp r0, #0x50    ; 比较 r0 是否大于 0x50
movgt r1, r1, #-0x10 ; 如果大于则 r1 减去 0x10
```

```
movgt r0, r1    ; 将修改后的 r1 送给 r0
```

8、编写一段 ARM 汇编程序，实现数据块复制，将 R0 指向的 8 个字的连续数据保存到 R1 指向的一段连续的内存单元。

答：

```
copy_loop:
   ldr r2, [r0], #4   ; 从地址 r0 加载一个字到 r2，同时 r0 加 4
   str r2, [r1], #4   ; 将加载的值存储到地址 r1，同时 r1 加 4
   subs r3, r3, #1    ; 计数器减一
   bne copy_loop      ; 如果计数器不为零，则继续循环
```

伪指令：

1. 在 GNU 风格的 ARM 汇编中如何定义一个全局的数字变量？

答：在 GNU 风格的 ARM 汇编中，定义一个全局的数字变量可以使用 .word 指令。例如：

```
.data
aaaa: .word 66
```

2. AAPCS 中规定的 ARM 寄存器的使用规则是什么？

AAPCS 规定了 ARM 处理器在函数调用时的寄存器使用规则，以确保不同函数能够正确传递参数、返回值，并保持一致的调用约定。根据 AAPCS 规范，R0 至 R3 用于传递函数参数，同时 R0 也用于存储返回值。如果参数超过四个，则通过栈进行传递，调用者需要自行管理超出部分的参数存储。

寄存器的保存策略分为两类：Caller-Saved 和 Callee-Saved。R0 至 R3 以及 R12（IP）属于 Caller-Saved 寄存器，调用函数之前，调用者需要手动保存这些寄存器的值，以防止它们在函数调用过程中被覆盖。而 R4 至 R11 属于 Callee-Saved 寄存器，被调用函数（Callee）必须在使用这些寄存器前保存其原始值，并在函数返回前恢复它们，以确保调用者的环境不被破坏。

此外，R13 作为栈指针（SP）用于管理函数调用时的栈空间，R14 作为链接寄存器（LR）存储返回地址，R15 作为程序计数器（PC）用于控制指令流。AAPCS 还要求栈地址必须 8 字节对齐，以优化内存访问效率，提高数据读取的性能。这些规则保证了不同程序和库之间的兼容性，使得 ARM 体系结构能够稳定、高效地执行函数调用。

3. 什么是内联汇编？什么是嵌入型汇编？两者之间的区别是什么？

答：内联汇编是指在 C 或 C++ 代码中直接插入汇编指令，通常使用 asm 或 __asm__ 关键字，适用于小规模优化。例如，__asm__("nop"); 可插入一个 NOP 指令。嵌入型汇编则是将汇编代码写在单独的 .s 或 .asm 文件中，并在 C 代码中用 extern 关键字调用，如 extern void my_function();。两者区别在于，内联汇编更灵活，可与 C 变量交互，而嵌入型汇编独立管理，适用于复杂底层优化。

4. 汇编代码中如何调用 C 代码中定义的函数？

答：可以通过声明函数的原型（prototype），然后直接调用函数名来实现。例如：

```
.global func

... ; 写汇编代码

bl func ; 调用 C 函数
```

六、实验结论：

在本次 ARM 汇编实验中，通过两个主要实验（ADD 和 ARM 伪指令实验），我们深入理解并掌握了 ARM 汇编语言的基本使用，以及如何通过使用汇编语言在 ARM 处理器上执行加法和其他基本操作。

[1]. **ARM 汇编实验（ADD）：** 通过编写一个简单的加法程序，成功地将寄存器 r0 和 r1 中的数据相加，并将结果存储回 r1。实验中使用的 mov、add 等基本指令使我们对 ARM 汇编的操作逻辑有了清晰的理解。通过单步仿真，我们能够观察到寄存器的变化，验证了程序的正确性，得到了预期的结果，即寄存器 r1 的值为 24。

[2]. **ARM 伪指令实验：** 在伪指令实验中，我们编写了一个程序来对存放在内存中的数据进行相加，并通过循环结构来处理数组元素。在该实验中，我们使用了 ldr、add、cmp、bne 等指令，并利用寄存器 r0 和 r4 完成数据加载和累加的任务。通过仿真，我们验证了寄存器 r4 最终保存了数组元素的和，结果为 51，符合预期。

总的来说，这两个实验增强了我们对 ARM 汇编语言的掌握，尤其是通过使用汇编语言操作寄存器和内存，熟悉了常用的指令、伪指令的应用，以及调试和仿真的基本方法。这为后续更复杂的嵌入式系统开发打下了坚实的基础。

七、指导教师批阅意见：

成绩评定：

指导教师签字：

年　　月　　日

八、备注：

注：报告内的项目或内容设置，根据实际情况可加以调整和补充，写不下可另附一页。